Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

# ЗВІТ

з лабораторної роботи №3
з дисципліни «Проектування програмних систем для мобільних пристроїв»

**«Мобільні технології та інструментарій»**

**Варіант №4**

Виконав:
студент 2 курсу, групи ІМ-51мн
Ковальов Олександр

Перевірив:
асистент, Нестерук А. О.

Дата здачі: 19.10.2026

КИЇВ – 2026

**Мета.** Вивчити основні засади програмування для мобільних пристроїв, провести їх класифікацію, опанувати основні середовища розробки додатків – Android Studio та PHPStorm.

**Завдання.** Додати у програму, розроблену у лабораторній роботі №2, можливості введення, виведення та редагування даних відповідно до варіанту. Під час запуску програма повинна зчитувати дані з файлу. При закінченні роботи треба зберігати дані у файл. Зі зміною стану життєвого циклу Activity записувати інформацію до журналу. Запустити програму на емуляторі.

**Завдання за варіантом.** Доменна область: Абітурієнт.

Поля: ID, Прізвище, Ім’я, По-батькові, Адреса, Телефон, Оцінки.

Створити масив об’єктів.

Вивести:

а. список абітурієнтів, які мають незадовільні оцінки;

б. список абітурієнтів, середній бал у яких вище заданого;

в. вибрати задане число `N` абітурієнтів, що мають найбільш високій середній бал (вивести також повний список абітурієнтів, які мають напівпрохідний бал).

## Хід роботи.

Для виконання роботи використано застосунок, розроблений у другій лабораторній роботі, без жодної зміни архітектури MVC чи базового стеку – Rust, Tauri, React з TypeScript. Повторно не розглядається введення та редагування абітурієнтів – це вже реалізовано в минулій роботі. Нижче описано лише те, що додано саме в межах цієї роботи: зчитування й збереження даних у файл, журналювання зміни стану Activity та видалення абітурієнтів.

Зчитування та запис даних реалізовано в новому модулі `storage`: функція `load` читає JSON-файл `applicants.json` і десеріалізує його в масив, а якщо файл відсутній або пошкоджений (наприклад, після першого встановлення застосунку), повертає початковий зразок із колекції в коді, попередньо залишивши попередження в журналі. Функція `save` серіалізує поточний масив абітурієнтів у форматований JSON і перезаписує файл. Він зберігається безпосередньо в каталозі даних застосунку – `app_data_dir()` на Android повертає `activity.dataDir.absolutePath`, корінь приватного каталогу застосунку, а не його підкаталог `files/`. Це підтверджено відповідною командою для його перегляду:

```
1 | adb shell run-as com.xairaven.lab3 cat applicants.json
```

Вище використовується утиліта Android Debug Bridge (ADB), яка дозволяє комунікувати хостовій системі з емулятором.

```
xairaven@laptop ~/Files/Programming/KPI-DSSMD ‹main›
$ adb shell run-as com.xairaven.lab3 cat applicants.json
[
  {
    "id": 1,
    "last_name": "Бондаренко",
    "first_name": "Олена",
    "patronymic": "Іванівна",
    "address": "м. Київ, вул. Хрещатик, 10",
    "phone": "+380501112233",
    "grades": [
      10,
      11,
      9,
      12
    ]
  },
  {
    "id": 2,
    "last_name": "Шевченко",
    "first_name": "Тарас",
    "patronymic": "Григорович",
    "address": "м. Львів, вул. Франка, 5",
```

Завантаження даних виконується в блоці `setup` основного модулю застосунку, ще до того, як репозиторій стає керованим станом Tauri – це гарантує, що жоден контролер не отримає доступ до сховища раніше, ніж воно буде заповнене даними з файлу.

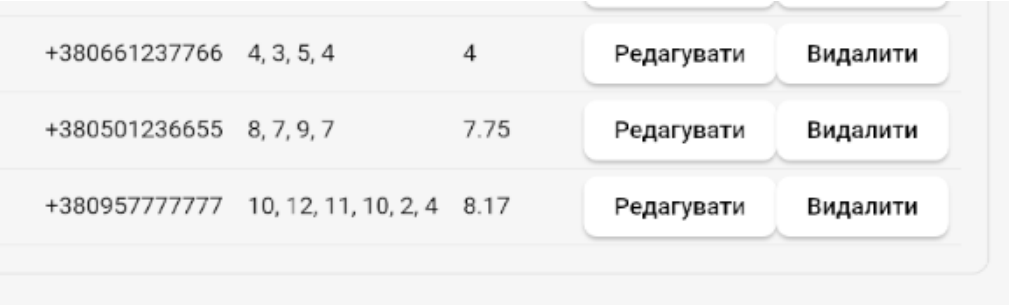
Для журналювання зміни стану життєвого циклу Activity підключено плагін `tauri-plugin-log` разом із крейтом `log`. Замість білдеру Tauri використано пару `build(...)` і замикання `app.run`, що дозволяє перехоплювати події після побудови застосунку. Оброблено події `Resumed` та `Suspended` (доступні лише під час запуску на мобільному пристрої) – це не довільна імітація, а справжні події, які Tauri генерує безпосередньо з `onResume`/`onPause` єдиної реальної Activity застосунку. `Exit` та `CloseRequested` оброблено як аналоги для десктопного запуску під час розробки. Збереження даних у файл викликається саме в обробниках `Suspended`/`Exit`/`CloseRequested`, а не після кожної зміни даних.

Коректність реалізації перевірено на емуляторі Android – під час згортання застосунку в журналі з'являється запис про `Suspended` й виконується збереження, під час розгортання – запис про `Resumed`. Перемикання між екранами (маршрутами `HashRouter`) жодного запису в журналі не створює, оскільки це зміна стану JavaScript у межах тієї самої Activity, а не подія життєвого циклу Android.

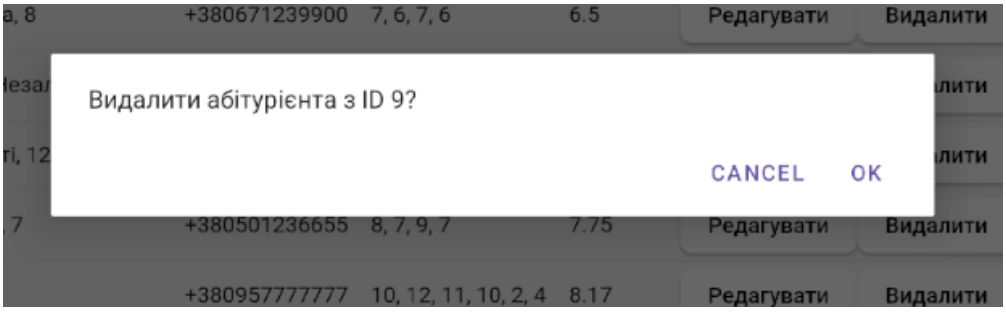
```
┌─xairaven@laptop ~/Files/Programming/KPI-DSSMD (main●)
└─$ adb logcat | grep -i "lifecycle"
10-19 18:25:53.970  8922  8922 D ProcessLifecycle63: androidx.startup is not available, initializing using ProcessLifecycleOwnerInitializer instead.
10-19 18:38:16.887  9420  9477 I lab3_lib: [lab3_lib] Activity lifecycle: onDestroy (Exit) - saving applicants
10-19 18:38:21.037  9574  9592 I lab3_lib: [lab3_lib] Activity lifecycle: window focus changed to true
10-19 18:38:31.427  9574  9592 I lab3_lib: [lab3_lib] Activity lifecycle: window focus changed to false
10-19 18:38:33.238  9574  9592 I lab3_lib: [lab3_lib] Activity lifecycle: window focus changed to true
10-19 18:38:37.493  9574  9592 I lab3_lib: [lab3_lib] Activity lifecycle: window focus changed to false
10-19 18:38:38.232  9574  9592 I lab3_lib: [lab3_lib] Activity lifecycle: onPause (Suspended) - saving applicants
10-19 18:38:57.138  9574  9592 I lab3_lib: [lab3_lib] Activity lifecycle: onResume (Resumed)
10-19 18:38:57.961  9574  9592 I lab3_lib: [lab3_lib] Activity lifecycle: window focus changed to true
^C
```

Персистентність перевірено повним циклом: абітурієнта відредаговано, застосунок згорнуто, після чого при повторному запуску зчитані з файлу дані відповідали останньому відредагованому стану, а не початковому зразку з коду.

Для завершеності операцій введення/виведення/редагування було реалізовано також видалення абітурієнта. В репозиторій додано метод `delete`, який повертає помилку, якщо абітурієнта з вказаним `ID` не існує. На боці Rust додано відповідну команду, на боці React – кнопку «Видалити».




При видаленні з'являється діалогове вікно для підтвердження.



Варто окремо зупинитися на тому, що саме мається на увазі під «станом життєвого циклу Activity» в цій роботі, оскільки в минулій роботі вже піднімалося питання кількох Activity. Реальна `Activity` в Android – це компонент, яким керує сама операційна система, а не код застосунку: екземпляр створюється викликом `startActivity(Intent)`, отримує власний запис у стеку задач (back stack), і проходить власний життєвий цикл `onCreate` → `onStart` → `onResume` → `onPause` → `onStop` → `onDestroy` (або `onRestart` → `onStart` → `onResume`, якщо користувач повертається до неї, не знищивши). Кожна Activity може бути незалежно знищена й відновлена системою (наприклад, при нестачі пам'яті чи повороті екрана), для чого існують `onSaveInstanceState`/`onRestoreInstanceState`.

Tauri-застосунок має рівно одну таку Activity (`MainActivity`, що монтує єдиний WebView) на весь час роботи програми – вона створюється один раз при запуску й знищується один раз при завершенні. Саме цю, цілком реальну, Activity журналює основний модуль у цій роботі. «Кілька Activity» реалізовано як структурну аналогію в межах цієї єдиної Activity: власний маршрут `HashRouter` (власна адреса та запис в історії навігації браузера), власний модуль-Controller і власний компонент-View на кожен екран – тобто відтворено розподіл відповідальності та відчуття «переходу вперед/назад», але не сам механізм: під час переходу між екранами жодного разу не викликається `startActivity()`, не створюється новий об'єкт Activity, не додається запис у реальний стек задач Android, і – що підтверджено журналом саме в цій роботі – жодна подія життєвого циклу Android не виникає.

Використання кількох справжніх Activity натомість додало б цій задачі зайву вагу: кожна Activity вимагала б власного дублювання обробки життєвого циклу, передачі даних між екранами лише через `Intent`-екстра чи статичний singleton (що так само довелося б захищати від паралельного доступу, як `Repository` в цій роботі), повільніші переходи (створення нового вікна й поверхні замість перемальовування вмісту) та складніше спільне тестування логіки, розкиданої по різних класах Activity. Саме тому сучасна Android-розробка (Jetpack Navigation Component, а згодом Jetpack Compose) відмовилася від «одна Activity на екран» на користь архітектури з єдиною Activity, що утримує навігаційний граф, а роль окремого екрана виконують Fragment або Composable-функція – тобто фактично той самий підхід, що й у цій роботі, лише замість Fragment/Composable використано React-компонент і маршрут `HashRouter`. Кілька реальних Activity лишаються доречним і єдино правильним рішенням для принципово незалежних точок входу (обробник `Intent` від іншої програми, deep link,

окремий модуль у великому проєкті) – але не для набору тісно пов’язаних екранів одного невеликого застосунку, як у цій роботі.

**Висновок.** У цій лабораторній роботі до застосунку «Абітурієнт», розробленого на MVC в минулій роботі, додано збереження стану між запусками – дані зчитуються з JSON-файлу під час старту (з запасним варіантом – початковим зразком, якщо файл відсутній чи пошкоджений) і записуються назад у файл при згортанні або завершенні роботи застосунку. Також реалізовано журналювання зміни стану життєвого циклу Activity, яке спирається на справжні події `onResume`/`onPause` єдиної Activity Tauri-застосунку, а не на довільну імітацію. Для завершеності операцій над даними додано видалення абітурієнта з підтвердженням. Коректність персистентності та журналювання перевірено на емуляторі Android: через `logcat` підтверджено появу записів про зміну стану лише при реальному згортанні/розгортанні/завершенні застосунку (а не при переході між екранами), а через консоль – що відредаговані дані дійсно зберігаються у файлі та коректно зчитуються при повторному запуску. Окремо обґрунтовано, чому єдина Activity, властива обраному стеку (Tauri), не є недоліком: вона відповідає підходу, якого сучасна Android-розробка сама дотримується останніми роками, залишаючи кілька справжніх Activity доречними лише для принципово незалежних точок входу в застосунок.

## Контрольні запитання.

1. **Класифікуйте мобільні пристрої та назвіть основні їх особливості.** Мобільні пристрої прийнято класифікувати за кількома ознаками, зокрема за призначенням і форм-фактором, де виділяють смартфони – універсальні пристрої з повноцінною ОС, стільниковим зв’язком та доступом до магазинів застосунків, планшети, що мають більший екран і здебільшого орієнтовані на споживання контенту й продуктивність (не завжди зі стільниковим модемом), та фаблети як проміжний форм-фактор між ними. Крім того, існують носимі пристрої (смарт-годинники, фітнес-трекери, AR/VR-гарнітури) з обмеженими ресурсами для автономної або допоміжної роботи зі смартфоном, електронні книги з енергоощадними e-ink дисплеями, а також вбудовані мобільні пристрої, такі як бортові комп’ютери, термінали оплати та IoT-рішення, які є вузькоспеціалізованими і часто не мають власного екрана чи повноцінного інтерфейсу. За операційною системою пристрої здебільшого поділяються на Android, iOS/iPadOS та менш поширені чи спеціалізо-

вані ОС (HarmonyOS, KaiOS, RTOS). Серед основних особливостей мобільних пристроїв варто виділити їхню портативність і компактність, автономне живлення від акумулятора, яке накладає жорсткі обмеження на енергоспоживання, сенсорний екран як основний спосіб введення та багатий набір вбудованих сенсорів (акселерометр, гіроскоп, GPS, камера, біометрія). Їм також притаманна наявність різноманітних каналів зв'язку (стільниковий, Wi-Fi, Bluetooth, NFC), порівняно обмежені обчислювальні ресурси та пам'ять, велика апаратна фрагментація через різноманіття моделей та версій ОС, і, зрештою, розповсюдження програмного забезпечення переважно через централізовані магазини застосунків (Google Play, App Store) з використанням суворих моделей дозволів (permissions) та ізольованого середовища (sandboxing) для кожної програми.

2. **Охарактеризуйте середовище розробки Android Studio.**

   Android Studio – це офіційне інтегроване середовище розробки (IDE) для платформи Android від Google, створене на базі IntelliJ IDEA від JetBrains. Серед його ключових можливостей – повноцінна підтримка Kotlin та Java, а також можливість роботи з нативним C/C++ кодом через Android NDK. Середовище використовує систему збірки на основі Gradle, що забезпечує гнучку конфігурацію залежностей, варіантів збірки та роботу з багатомодульними проєктами. Вбудований менеджер віртуальних пристроїв (AVD Manager) разом з емулятором дозволяє запускати й тестувати застосунки на різних версіях ОС і розмірах екранів без необхідності мати фізичний девайс. Розробникам доступний візуальний редактор макетів (Layout Editor) із підтримкою ConstraintLayout, попереднім переглядом у реальному часі та одночасним редагуванням XML і Jetpack Compose. Інтелектуальний редактор коду пропонує автодоповнення, рефакторинг та статичний аналіз із лінтером. Для оптимізації застосунків використовуються інструменти профілювання (Android Profiler), що аналізують навантаження на CPU, пам'ять, мережу та енергоспоживання, а APK/Bundle Analyzer допомагає контролювати розмір фінальної збірки. Також передбачена пряма інтеграція з `adb` та `Logcat` для перегляду журналів у реальному часі, вбудовані утиліти для роботи з ресурсами (наприклад, Vector Asset Studio) та зв'язок із хмарними сервісами на кшталт Firebase. Оскільки Android Studio побудована на платформі IntelliJ, вона повністю успадковує її екосистему плагінів, потужну систему контролю версій, зручну навігацію по коду та просунутий дебагер.

# Лістинг.

## lib.rs

```rust
mod applicant;
mod repository;
mod sample;
mod screen;
mod storage;

use repository::Repository;
use std::sync::{Mutex, PoisonError};
use tauri::{AppHandle, Manager, RunEvent, WindowEvent};

const DATA_FILE_NAME: &str = "applicants.json";

#[cfg_attr(mobile, tauri::mobile_entry_point)]
pub fn run() {
    let app = tauri::Builder::default()
    .plugin(tauri_plugin_opener::init())
    .plugin(tauri_plugin_log::Builder::new().build())
    .setup(|app| {
        let data_dir = app.path().app_data_dir()?;
        std::fs::create_dir_all(&data_dir)?;

        let applicants = storage::load(&data_dir.join(DATA_FILE_NAME));
        app.manage(Mutex::new(Repository::from_applicants(applicants)));

        Ok(())
    })
    .invoke_handler(tauri::generate_handler![
    screen::failing::get_failing,
    screen::threshold::get_above_threshold,
    screen::top::get_top_n,
    screen::students::list_applicants,
    screen::students::add_applicant,
    screen::students::update_applicant,
    screen::students::delete_applicant,
    ])
    .build(tauri::generate_context!());

    let app = match app {
        Ok(app) => app,
        Err(err) => {
            eprintln!("Error occurred while building Tauri application: {err}");
            std::process::exit(1);
        },
    };

    app.run(|app_handle, event| match event {
        RunEvent::Exit => {
            log::info!("Activity lifecycle: onDestroy (Exit) — saving applicants");
            persist(app_handle);
        },
        // WindowEvent::Suspended/Resumed are the real Activity lifecycle —
        // Tauri dispatches them straight from onPause()/onResume() on
        // Android, unlike Lab 2's HashRouter, which only simulates one.
        #[cfg(mobile)]
        RunEvent::WindowEvent {
            event: WindowEvent::Resumed,
            ..
        } => {
            log::info!("Activity lifecycle: onResume (Resumed)");
        },
```

```
        #[cfg(mobile)]
        RunEvent::WindowEvent {
            event: WindowEvent::Suspended,
            ..
        } => {
            log::info!("Activity lifecycle: onPause (Suspended) — saving applicants");
            persist(app_handle);
        },
        RunEvent::WindowEvent {
            event: WindowEvent::Focused(focused),
            ..
        } => {
            log::info!("Activity lifecycle: window focus changed to {focused}");
        },
        RunEvent::WindowEvent {
            event: WindowEvent::CloseRequested { .. },
            ..
        } => {
            log::info!("Activity lifecycle: CloseRequested — saving applicants");
            persist(app_handle);
        },
        _ => {},
    });
}

fn persist(app_handle: &AppHandle) {
    let Some(state) = app_handle.try_state::<Mutex<Repository>>() else {
        log::error!("Failed to save: repository state is not found");
        return;
    };

    let Ok(data_dir) = app_handle.path().app_data_dir() else {
        log::error!("Failed to save: app data directory not found");
        return;
    };

    let applicants = state
    .lock()
    .unwrap_or_else(PoisonError::into_inner)
    .list()
    .to_vec();

    if let Err(err) = storage::save(&data_dir.join(DATA_FILE_NAME), &applicants) {
        log::error!("{err}");
    }
}
```

## storage.rs

```
//! Persisting applicants to a JSON file in the app's data directory.
//!
//! Loaded once on startup (falling back to the Lab 2 sample seed if the
//! file is missing or unreadable) and saved once on suspend/exit

use crate::applicant::Applicant;
use crate::sample;
use std::path::Path;

pub fn load(path: &Path) -> Vec<Applicant> {
    let result = std::fs::read_to_string(path)
    .map_err(|err| err.to_string())
    .and_then(|contents| {
        serde_json::from_str(&contents).map_err(|err| err.to_string())
```

```
    });

    match result {
        Ok(applicants) => applicants,
        Err(err) => {
            log::warn!(
            "Failed to load {path}: {err}. Using initial data.",
            path = path.display(),
            );
            sample::get_applicants()
        },
    }
}

pub fn save(path: &Path, applicants: &[Applicant]) -> Result<(), String> {
    let json = serde_json::to_string_pretty(applicants)
    .map_err(|err| format!("Failed to serialize applicants: {err}"))?;

    std::fs::write(path, json)
    .map_err(|err| format!("Failed to write {}: {err}", path.display()))
}
```